

$\pi 0$ 的核心思路

把两件事合在一起：

VLM (视觉语言模型)

- 理解“帮我叠衣服”是什么意思
- 看懂衣服在哪、怎么抓

大脑思考

Flow Matching (流匹配)

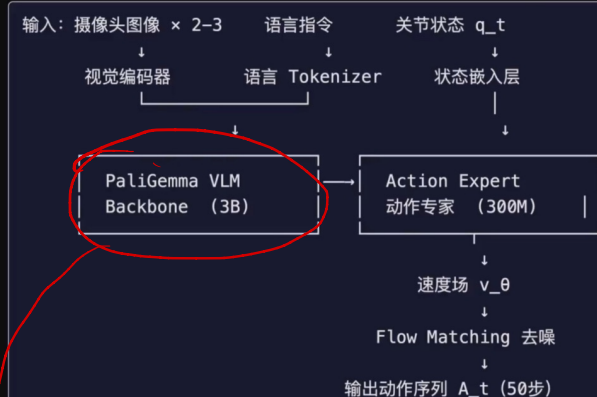
- 生成连续的关节角度值 (浮点数)
- 每秒输出 50 次控制指令

小脑运动

就像给一位已经“见多识广”的大脑 (VLM)，加上一双经过专门训练的手 (Flow Matching 动作输出)。

模型总参数：3.3B (3B VLM 骨干 + 300M 动作专家)

模型架构总览



三类输入，两个世界

$\pi 0$ 把所有输入分成两类，分别处理：

语义世界 (VLM 骨干处理)

- 图像 token: 通过视觉编码器 → 投影到 embedding 空间
- 语言 token: 正常的文字 token

控制世界 (动作专家处理)

- 关节状态 q_t : 机器人当前的关节角度
- 带噪声动作 A_t^ϵ : 从高斯噪声出发，逐步去噪

两个世界通过 Self-Attention 相互交流：VLM 骨干“告诉”动作专家当前场景；动作专家“问”VLM 下一步该怎么动。

二、什么是 PaliGemma？（pi0 的“多模态基座”）

最直接的定义：

PaliGemma = SigLIP（视觉编码器） + Gemma（语言模型） + 一个简单的投影层（Projection Layer）。

它是 Google 开源的一个极其强大的 视觉-语言大模型（VLM）。

面试时，描述 PaliGemma 工作原理的满分话术：

“PaliGemma 的架构非常简洁优雅。

- 当机器人拿到一张图片时，先扔给 **SigLIP**，SigLIP 把图片切成一个个 Patch，提取出高维的**视觉特征 Token**。
- 这些视觉 Token 会通过一个简单的线性映射层（Projector），把维度对齐到语言模型的维度。
- 然后，模型把**视觉 Token 和用户的指令（文本 Token）拼接在一起，组成一段超长的序列**。
- 最后，把这个混合序列送进 **Gemma** 大脑中。Gemma 就能结合看到的画面和听到的指令，输出对环境的多模态理解（在 pi0 中，这些理解被输出为 Condition 向量，喂给动作专家）。”

面试终极拷问：为什么 pi0 要选 PaliGemma 作为基座，而不是 LLaVA 或 Qwen-VL？

如果面试官问出这个问题，你可以用下面这段话“绝杀”，展现你对机器人大模型选型的深刻理解：

你的满分回答：

“现在的开源 VLM 很多，比如 LLaVA、Qwen-VL。但是 Physical Intelligence（pi0的团队）选择基于 PaliGemma 架构，我认为有三个极其关键的工程和算法考量：

第一，空间几何认知极值（Spatial Awareness）。

很多 VLM（比如早期的 LLaVA）为了做图文聊天，预训练数据全是“给图片写描述（Captioning）”。但 PaliGemma 在预训练时，Google 故意喂了海量的**目标检测（Bounding Box）、图像分割、深度估计**的数据。这意味着 PaliGemma 的骨子里不仅知道“那是一个杯子”，还极其精准地知道杯子的边缘在哪、三维坐标大概是多少”。这对于需要做毫米级精准抓取的机器人来说，是不可替代的优势。

第二，高分辨率与多视角的适配能力。

机器人通常有全局相机和腕部相机，PaliGemma 配合 SigLIP 的特征提取，能够很好地处理多分辨率、多视角的图像输入，并且将这些视觉 Token 高效地压缩后塞给 Gemma。

第三，参数层极度舒适（3B 级）。

PaliGemma 的总参数量大概在 30 亿（3B）左右（SigLIP ~400M + Gemma ~2B + 杂项）。在真实的机器人部署中，太大的模型（如 7B/14B）推理延迟（Latency）会超过 100ms，导致控制系统发散（机械臂乱抖）；而太小的模型又缺乏物理常识。**3B 是目前单张卡训练、且能在机器人工控机上跑到 20Hz-50Hz 实时控制的完美甜点位。**”

PaliGemma 是一个优秀的‘小脑+半个大脑’，适合做精细控制；Qwen-VL 是一个聪明的‘最强大脑’，适合做高层语义规划（High-level Planning）。”

动作专家 (Action Expert)

为什么需要专门的动作专家?

直接用 VLM 输出动作有两个问题:

1. VLM 的 token 是离散的, 关节角度是连续的
2. VLM 在文字上训练, 让它直接输出高频控制信号会"破坏"语言知识

格式不匹配

灾难性遗忘

动作专家的设计

- 参数量 **300M** (远小于 3B 的 VLM 骨干, 降低推理开销)
- 使用 **独立的权重** —— 不与 VLM 共享参数
- 处理: 关节状态 q_t + 带噪动作块 A_t^r
- 输出: **速度场** v_θ , 指向"正确动作"的方向

实验验证: 动作专家使用独立权重比共享权重性能更好。

Attention 的设计: 三段式因果掩码

Token 类型: [图像, 语言] [关节状态] [动作序列]
Block 1 Block 2 Block 3

↑ 双向 Attention (Block 内部)

Block 1 → Block 2 → Block 3

↑ 单向 (前面的 block 不能看后面的)

单向信息流

设计逻辑:

- 图像和语言之间互相关注 (双向)
- 关节状态可以"回望"图像和语言
- 动作序列可以"回望"所有前面的信息
- **防止信息泄露**: 动作序列不能影响语言理解

图像不能看关节与动作

这保证了 VLM 的语义理解不被机器人控制信号干扰。

Flow Matching 第一步: 构造一条直线路径

目标: 从随机噪声 ε 出发, 走到正确动作 A_t 。

最简单的想法——在两者之间定义一条**线性插值路径**:

高斯噪声

$$A_t^r = \tau \cdot A_t + (1 - \tau) \cdot \varepsilon \quad \tau \in [0, 1]$$

以一个关节角度为例 ($A_t = 0.8$, $\varepsilon = -0.3$):

τ	计算	A_t^r	含义
0.0	$0 \times 0.8 + 1 \times (-0.3)$	-0.30	纯噪声
0.5	$0.5 \times 0.8 + 0.5 \times (-0.3)$	0.25	中间状态
1.0	$1 \times 0.8 + 0 \times (-0.3)$	0.80	目标动作

τ 从 0 走到 1, 动作就从噪声变成了目标。

Flow Matching 第二步：速度 = 对路径求导

路径 $A_t^\tau = \tau A_t + (1 - \tau)\varepsilon$ 定义好了，那沿着这条路走的速度是多少？

对 τ 求导：

$$\frac{dA_t^\tau}{d\tau} = A_t - \varepsilon$$

这就是真实速度：从噪声指向目标的方向向量。

用上面的例子验证：

$$\text{真实速度} = A_t - \varepsilon = 0.8 - (-0.3) = 1.1$$

关键性质：因为路径是直线，速度是常数——无论走到哪里，方向和大小都不变。

这就是 Flow Matching 比 Diffusion 只需 10 步的原因：直线不需要矫正方向。

快

Flow Matching 第三步：让模型学习这个速度

训练时的问题：推理时我们只有噪声 ε ，不知道目标 A_t ，也就不知道真实速度 $A_t - \varepsilon$ 。

解决方案：训练一个网络 v_θ ，输入带噪动作 A_t^τ 、**当前时间步 τ** 和场景观测 o_t ，预测速度：

$$\mathcal{L}(\theta) = \mathbb{E}_\tau \|v_\theta(A_t^\tau, \tau, o_t) - (A_t - \varepsilon)\|^2$$

训练过程：

1. 随机采一个真实动作 A_t ，采一个噪声 ε
2. 随机采一个时间步 τ ，计算带噪动作 $A_t^\tau = \tau A_t + (1 - \tau)\varepsilon$
3. 计算真实速度 $A_t - \varepsilon$
4. 把 (A_t^τ, τ, o_t) 喂给网络，预测速度，最小化均方误差

训练结束后，网络面对任意噪声状态，都能预测“应该朝哪走”。

Flow Matching 学到的速度场本质是“指向数据流形”的引力场。

Flow Matching 第四步：推理——沿速度场走 10 步

推理时，从 $A_t^0 = \varepsilon$ （纯噪声）出发，每步用模型预测速度，做欧拉积分：

$$A_t^{\tau+\delta} = A_t^\tau + \delta \cdot v_\theta(A_t^\tau, \tau, o_t)$$

步长 $\delta = 0.1$ ，走 10 步， τ 从 0 走到 1，动作从纯噪声收敛到目标值。

整个过程在 RTX 4090 上约 **27ms**，输出 50 步动作序列（1 秒运动）。

Flow Matching vs Diffusion：为什么更快？

	Diffusion	Flow Matching
加噪方式	逐步加高斯噪声（前向过程）	线性插值 $\tau A_t + (1 - \tau)\varepsilon$
训练目标	学习逐步去噪的反向过程	直接学习速度场 $A_t - \varepsilon$
路径形状	弯曲（每步方向不同）	直线（方向恒定）
推理步数	100-1000 步	10 步

“反向过程”是什么意思：Diffusion 先把数据加噪成纯噪声（前向），再训练模型一步步去掉噪声（反向）。反向过程每一步依赖上一步的结果，路径是弯曲的随机游走，方向不断变化，所以需要上百步才能收敛。

Flow Matching 为什么不需要：路径是直线，速度 $A_t - \varepsilon$ 是常数，每一步方向都相同，10 步走完全程。

对机器人来说：50Hz 控制每 20ms 一帧，73ms 的总推理延迟已经很紧张——10 步而非 1000 步，是实用部署的关键。

核心贡献总结

1. 架构创新

将 3B 参数的 PaliGemma VLM 与 300M 动作专家结合，通过三段式因果掩码保护语言知识，同时实现高频精确控制。

2. 数据规模

约 10,000 小时、8 种机器人平台、68 种任务的最大规模机器人演示数据集。

3. 跨平台泛化

一套模型权重，统一控制从 6 轴单臂到移动双臂的全部机器人。

4. 任务复杂度

首次在乱堆衣物叠放等高难度长程任务上取得实质性成功。

局限性与未来方向

当前局限

- 推理延迟约 **73ms**，对极高速任务（如插插头）仍有挑战
- 训练数据仍以固定环境为主，**开放世界泛化**尚未完全解决
- **长时记忆**缺失：无法记住几分钟前发生的事
- 自我修正能力弱：依赖数据覆盖，缺少主动探索

π 团队后续工作（本系列后续各期）

- EP05 π0.5：开放世界泛化（陌生家庭也能用）
- EP08 π*0.6：RL 自我改进（从失败中学习）
- EP12 MEM：长期短期记忆架构
- EP14 π0.7：真正的通用可控模型